

BLOC 1 — FRONT-END

Support d'oral (10 minutes) Ce qui est attendu par le jury et comment j'y réponds

Projet Airlines Reservation — Flask + PostgreSQL. Chaque point cite le fichier et la ligne de code correspondante, pour pouvoir l'ouvrir en direct devant le jury si besoin.

MINUTAGE CONSEILLÉ (10 MIN)

0:00	Intro projet + structure HTML/CSS (C1.a, C1.d)
1:30	Responsive mobile/tablette/desktop (C1.b)
3:00	Accessibilité & inclusion : clavier, lecteurs d'écran, dyslexie (C1.c)
5:30	Référencement naturel / SEO (C1.e)
7:00	JavaScript : interactivité, validation, API, librairie externe (C2.a-d)
9:30	Conclusion + points de vigilance assumés

C1.a Intégrer une maquette en HTML/CSS propre et structuré

ATTENDU DU JURY

Un HTML valide et sémantique (header, nav, main, footer), des composants répétés (cartes hôtel, cartes vol) qui suivent une structure cohérente plutôt que d'être recopiés différemment sur chaque page.

CE QUE J'AI FAIT

Un seul système de composants (`.h-card` pour les hôtels, `.v-card` pour les vols) réutilisé à l'identique sur l'accueil, la liste, la recherche et la réservation. J'ai aussi corrigé un bug de balisage présent sur 16 pages : le bouton de connexion était un `<button>` contenant un `<a>` mal fermé — invalide en HTML et cassant la navigation au clavier. Remplacé par un simple lien.

static/css/styles.css : ligne 2258 – définition de `.h-card`

templates/index.html : ligne 53 – `<a href="/login" class="boutton_connect">`

À dire : « J'ai unifié toutes les cartes du site autour d'un seul système de composants, et corrigé un bug de balisage invalide qui cassait la navigation au clavier sur 16 pages. »

C1.b Rendre le site responsive (mobile / tablette / desktop)

ATTENDU DU JURY

Un affichage correct sur mobile, tablette et desktop, sans débordement ni contenu tronqué, avec une navigation adaptée à chaque taille d'écran. 0/3 au 1er passage → corrigé intégralement

CE QUE J'AI FAIT

Trouvé la cause racine : une règle globale `display:flex` empêchait `<main>` de rétrécir sous la largeur de son contenu le plus large (un tableau, par exemple), ce qui faisait déborder toute la page sur mobile. Une fois corrigé, j'ai retravaillé un par un : le menu (9 liens → 3 visibles + bouton « voir plus »), les tableaux RGPD (scroll horizontal contenu dans leur cadre), et les images (version allégée et recadrée servie sous 540px via `<picture>`). Vérifié à 1920px, 768px et 375px.

static/css/styles.css : ligne 1354 – `body > main { flex: 1 1 auto; min-width: 0; }`

static/css/styles.css : ligne 351 – `.nav-more-toggle` (menu mobile repliable)

static/css/styles.css : ligne 2074 – `.legal-table-wrap { overflow-x: auto; }`

templates/index.html : ligne 111 – `<picture><source media="(max-width: 540px)">`

À dire : « Ce point valait 0/3 au premier passage. J'ai remonté à la cause racine — une règle flex globale — plutôt que de corriger page par page, puis vérifié sur 3 largeurs d'écran. »

C1.d Logique d'intégration réutilisable (système de classes)

ATTENDU DU JURY

Que le CSS suive une organisation reconnue plutôt que des noms de classes improvisés au fil de l'eau.

CE QUE J'AI FAIT

Nomenclature BEM (Block-Element-Modifier) pour tous les nouveaux composants : `.h-card__body` , `.v-card__footer` , `.admin-sidebar__nav` . Feuille de style organisée en sections commentées par fonctionnalité (accueil, panier, admin, accessibilité, carrousel...).

À dire : « J'ai suivi la convention BEM : le nom de la classe indique à lui seul à quel composant et à quelle sous-partie elle appartient, ce qui évite les conflits de style. »

 Ce qui est attendu par le jury

Que le site reste utilisable par des publics ayant des besoins spécifiques : troubles de la lecture (dyslexie), navigation au clavier uniquement, utilisateurs de lecteurs d'écran (non-voyants / malvoyants).

 Non-voyants / lecteurs d'écran

- **Alt text réel**, jamais générique : `alt="{ { hotel.nom } }"` décrit vraiment l'image, pas `alt="image1"`.
- **Labels reliés aux champs** via `for=/id=` sur tous les formulaires (login, contact, paiement...).
- **Boutons icône-seule** décrits par `aria-label` : ex. `aria-label="Voir plus de liens"` sur le menu mobile replié.
- **Menus dynamiques** exposés à l'assistance : `aria-haspopup="true"` + `aria-expanded` mis à jour en JS quand le menu « Mon compte » s'ouvre/se ferme.
- **Structure sémantique** : `<header>`, `<nav>`, `<main>`, `<footer>` sur toutes les pages, `lang="fr"` sur `<html>`.
- L'info « Hôtel »/« Vol » sur les badges n'est jamais donnée **uniquement** par la couleur : le mot est toujours écrit à côté.

```
templates/index.html : lignes 40-42 -  
aria-haspopup, aria-expanded
```

```
templates/index.html : ligne 100 - aria-label="Voir plus de liens"
```

```
templates/hotels.html : ligne 123 -  
alt="{ { hotel.nom } }"
```

 Dyslexie

- Bouton flottant présent sur **toutes** les pages, qui bascule vers une police adaptée (**OpenDyslexic**).
- Le choix est mémorisé en `localStorage` : il reste actif d'une page à l'autre.
- La police ne s'applique **qu'au texte** — jamais aux icônes Font Awesome, qui utilisent leur propre police de glyphes et deviendraient illisibles sinon.

```
static/js/script.js : ligne 5 -  
initDyslexiaToggle()
```

```
static/css/styles.css : ligne 3301 -  
@font-face 'OpenDyslexic'
```

```
static/assets/fonts/opendyslexic-regular.woff2 (+ bold)
```

Navigation au clavier

- Tous les éléments cliquables sont de **vrais** éléments interactifs (`<a>` , `<button>` , `<select>`) — jamais de `<div onclick>` , donc naturellement atteignables et activables au clavier (Tab / Entrée) sans bidouille `tabindex` .
- Le menu « Mon compte » et la modale admin se ferment avec la touche **Échap**, en plus du clic à l'extérieur.
- Le curseur `cursor: pointer` qui s'affichait sur toute une carte hôtel/vol (alors que seul le bouton « Réserver » est cliquable) a été retiré — il induisait en erreur, y compris les utilisateurs clavier qui s'y fient pour repérer les zones interactives.
- **Audit du jour** j'ai contrôlé chaque bouton icône-seule du site : 3 boutons (`account-menu__toggle` , `admin-icon-btn` , `admin-modal__close`) supprimaient l'indicateur de focus (`outline: none`) sans le remplacer — invisibles au clavier. Ajout d'un contour visible via `:focus-visible` sur les trois.

`static/js/script.js` : lignes 104-109 – fermeture au clavier (`Escape`) du menu compte

`templates/admin_comptes.html` : lignes 207-209 – fermeture au clavier de la modale

`static/css/styles.css` : `.account-menu__toggle:focus-visible` , `.admin-icon-btn:focus-visible` , `.admin-modal__close:focus-visible`

À dire : « L'accessibilité n'est pas traitée comme un ajout en fin de projet : les vraies balises interactives donnent la navigation clavier gratuitement, j'ai ajouté une police dyslexie mémorisée, des alt et aria-label descriptifs pour les lecteurs d'écran, et j'ai fait un dernier audit qui m'a permis de corriger 3 boutons dont le focus clavier était invisible. »

 Ce qui est attendu par le jury

Que chaque page ait un titre unique, une hiérarchie de titres logique, des métadonnées correctes, et n'affiche jamais de contenu vide ou cassé (hiérarchisation des contenus, sémantique, poids des pages...).

CRITÈRE	ÉTAT SUR LE SITE	RÉFÉRENCE
Titre unique par page	21 pages, chacune avec un <code><title></code> descriptif (ex. « Nos Hôtels Airlines Reservation »)	<code>templates/*.html</code> : ligne 8-9
Meta description unique	Sur toutes les pages publiques (accueil, hôtels, vols, contact, légales...)	<code>templates/index.html</code> : ligne 7
Balise canonique	<code><link rel="canonical"></code> pour éviter le contenu dupliqué	<code>templates/index.html</code> : ligne 9
Robots différenciés	<code>noindex</code> , <code>nofollow</code> sur les pages privées (login, panier, profil, admin) ; pages publiques indexables	<code>templates/login.html</code> : ligne 8
Sémantique HTML5	<code><header></code> , <code><nav></code> , <code><main></code> , <code><footer></code> , <code><section></code> sur toutes les pages	<code>templates/index.html</code>
Alt renseignés dynamiquement	Le texte alternatif reprend la vraie donnée (nom d'hôtel, ville) plutôt qu'un texte générique — utile aussi pour Google Images	<code>templates/hotels.html</code> : ligne 123
Dimensions d'image renseignées	<code>width</code> / <code>height</code> sur l'image héro pour éviter les décalages de mise en page au chargement (CLS)	<code>templates/index.html</code> : ligne 113
Bug corrigé	Le titre des cartes vol s'affichait vide : le code lisait <code>vol.nom</code> , une colonne qui n'existe pas en base. Remplacé par <code>vol.ville</code> , la vraie donnée disponible via la jointure avec la table destinations.	<code>templates/vols.html</code> : ligne 134

 Limites assumées (si le jury pose la question)

- Pas de **données structurées schema.org** (JSON-LD) sur les fiches hôtel/vol : amélioration identifiée mais non réalisée faute de temps.
- Pas de `sitemap.xml` ni de `robots.txt` à la racine : à ajouter pour un site en production réelle.

À dire : « Chaque page a un titre et une description uniques, les pages privées sont explicitement exclues de l'indexation, et j'ai corrigé un bug où les titres des cartes vol s'affichaient vides à cause d'une mauvaise colonne en base. Je suis conscient qu'il manque des données structurées schema.org et un sitemap — ce serait la suite logique. »

C2.a Enrichir l'interface (interactivité, animations)

JavaScript natif, sans framework : système de notifications (toast) générique réutilisé partout (succès formulaire, erreur, rate-limit), menu mobile repliable, options de recherche repliables, bandeau cookies RGPD.

```
static/js/script.js : ligne 47 – navToggle (navigation mobile)
```

```
static/js/script.js : ligne 59 – searchToggle (options de recherche)
```

C2.b Valider les saisies avec des méthodes adaptées

Sur le formulaire de paiement, les champs numéro de carte / expiration / CVV filtrent les caractères invalides **en temps réel** et formatent l'affichage automatiquement (espace tous les 4 chiffres, slash pour la date).

```
templates/paiement.html : oninput="this.value = this.value.replace(/[^\d-]/g, '')..."
```

C2.c Dialoguer avec le serveur sans recharger la page (API)

Gestion du compte en AJAX avec de **vrais verbes HTTP** : modification du profil en **PUT**, changement de mot de passe en **PATCH** (modification partielle), suppression de compte en **DELETE**. Le dashboard admin suit la même logique pour gérer les comptes clients. Chaque requête transmet le jeton CSRF via l'en-tête **X-CSRFToken**.

```
templates/profil.html : ligne 200 – sendRequest('/profil', 'PUT', payload)
```

```
templates/profil.html : ligne 184 – 'X-CSRFToken': CSRF_TOKEN
```

```
templates/admin_comptes.html : ligne 152 – apiRequest(), fonction fetch générique
```

C2.d Optimiser avec une librairie JS externe

Intégration de **Swiper.js** (v11.2.10) pour le carrousel « Nos coups de cœur » de l'accueil : swipe tactile mobile, navigation clavier, pagination et flèches — plutôt que de recoder ce comportement à la main. Fichiers téléchargés et hébergés en local (pas de dépendance à un CDN externe le jour de l'examen).

```
templates/index.html : ligne 355 – new Swiper('.destinations-swiper', {...})
```

```
templates/index.html : ligne 367 – keyboard: { enabled: true }
```

```
static/vendor/swiper/swiper-bundle.min.js
```

✓ Ce qu'il faut retenir

- Le point le plus critique du premier passage (**responsive : 0/3**) a été corrigé à la racine, pas page par page.
- L'accessibilité est traitée à plusieurs niveaux : structure sémantique, clavier, lecteurs d'écran, dyslexie — avec un audit final qui a permis de trouver et corriger 3 boutons invisibles au clavier.
- Le SEO technique (titres, descriptions, canonicals, robots, alt) est en place ; les données structurées et le sitemap restent une amélioration identifiée.
- Le JavaScript va au-delà des formulaires HTML classiques : vraies API REST (PUT/PATCH/DELETE), et une librairie externe choisie et justifiée plutôt que recodée.